Research Paper

Project Title: Shannon Entropy and Huffman Coding: The Greedy Foundation

Instructor: Dr. Tania Sultana, Assistant Professor, East West University

Team Members

#	Full Name	Role
1	Md. Samiul Hasan Bhuiyan	Team Lead
2	Md. Habibul Islam	Analysis & AI Lead
3	Jobaer Abduallh	Writing Lead
4	Abu Talha Swargo	Literature Manager

Department of Computer Science & Engineering
East West University, Dhaka, Bangladesh

Abstract

This paper studies how to make data files smaller using Shannon Entropy and Huffman Coding. We reviewed 20 research papers from IEEE and Google Scholar. The main problem is that some compression methods are slow or use too much power. We looked at Static Huffman and Adaptive Huffman algorithms. Our findings show that Static Huffman is good for saving files like JPEG because it is simple, but it needs to read data two times. Adaptive Huffman is better for live video because it

updates the code tree instantly. However, Adaptive Huffman uses a lot of CPU power. We made a table to compare their speed and memory use. We also found three research gaps. Most studies do not look at modern GPUs or small IoT sensors. There is also a lack of comparison between Huffman and new systems like ANS. These findings will help researchers build better data systems in the future.

1. Introduction

1.1 Motivation and Background *(by Abu Talha Swargo)*

In today's digital world, we use the internet for almost everything like watching videos, using social media, storing files in the cloud, and making video calls. All of these things create and transfer a very large amount of data every second. If this data is not reduced in size, it would take much more time to send, use a lot of storage, and make systems slow. That is why data compression is very important in real life.

The basic idea behind compression comes from Claude Shannon, who introduced something called entropy. In simple terms, entropy tells us the smallest number of bits needed to store information without losing anything. It shows the best possible limit for compression that we can never go below.To get close to this limit, a method called Huffman coding is used. It works in a simple way: it gives shorter codes to data that appear more often and longer codes to data that appears less often. This helps reduce file size while still keeping all the information safe. Because it is simple and works well, Huffman coding is used in many real systems today.

1.2 Problem Statement *(by Abu Talha Swargo)*

Static Huffman coding doesn't work well for streaming data because it assumes symbol frequencies stay the same, which isn't true in real life. Data keeps changing, so its efficiency can drop.Adaptive Huffman coding solves this by updating the tree as new data comes in, so it works better for real-time streams without needing a second pass. However, constantly updating the tree adds extra work for the computer, which can slow things down.This survey compares both methods, looks at how close they get to Shannon's entropy limit, and discusses the trade-off between efficiency and speed.

1.3 Research Questions

RQ1. How does Shannon's entropy lower bound formally define the theoretical minimum achievable compression ratio for any lossless algorithm, and how close does static Huffman coding come to achieving this bound in practice?

RQ2. What is the mathematical proof that Huffman coding produces an optimal prefix-free code — specifically, how does the greedy tree construction guarantee minimum expected code length compared to any alternative prefix-free assignment?

RQ3. How does Adaptive Huffman coding improve compression performance for real-time streaming compared to static Huffman, and what additional time and space complexity does the dynamic tree update procedure introduce?

1.4 Paper Organisation *(by Md. Habibul Islam)*

The remainder of this paper is organised as follows. Section 2 provides the theoretical background on Shannon entropy and Huffman coding. Section 3 describes the methodology used to conduct this

systematic review. Section 4 presents the review findings and analysis. Section 5 discusses research gaps and open problems. Section 6 concludes the paper.

2. Background

2.1 Algorithmic Paradigm Overview *(by Md. Habibul Islam)*

Huffman coding is a greedy algorithm that makes the best choice at every step to solve a problem. Specifically, it picks the two symbols that appear the least to build a tree from the bottom up [9]. To work correctly, these algorithms require the greedy choice property and optimal substructure [1][17]. This method is considered the most efficient way to shorten codes and save storage space [10]. Many academic texts use an exchange argument to prove that this specific greedy strategy always produces the optimal result for data compression [1][17].

2.2 Core Algorithm Definitions *(by Md. Habibul Islam)*

2.2.1 Static Huffman Coding

Static Huffman Coding is a technique used to compress data without losing any information. It works by counting how often each symbol appears in a file. The core step is building a binary tree from the bottom up, where the most frequent symbols are placed near the top with the shortest binary codes [1][9]. The output is a compressed file consisting of these variable-length codes.

Time Complexity: O(nlogn). This comes from the time needed to sort the n symbols by frequency and build the priority queue for the tree [1][9].

Space Complexity: O(n). This is required to store the frequency table and the tree structure for an alphabet of n symbols [1][9].

2.2.2 Adaptive (Dynamic) Huffman Coding

Adaptive Huffman Coding is a version that compresses data in a single pass. Unlike the static version, it does not need to know the frequencies before starting. The core step involves updating the tree every time a new symbol is processed using the Sibling Property [2][6]. The input is a live stream of data, and the output is a continuous stream of updated codes. Time Complexity: O(nlogn). Here, k is the total number of symbols in the message and L is the maximum bits per symbol. Each update takes O(L) time [2][6].

Space Complexity: O(n) [1][2][6]. Similar to the static version, the algorithm must maintain a dynamic tree in memory for an alphabet of size n [2][6].

2.3 Application Domain Primer *(by Jobaer Abdullah)*

Huffman coding and entropy-based compression techniques are really important in the way we communicate and store information. We have a lot of data like text and images and audio and video that needs to be sent over networks or stored on devices with space . To do this we use something called lossless compression algorithms, including Huffman coding,which helps make files smaller without losing any of the information .

For instance when we zip files or use image formats like PNG these formats use a kind of coding to reduce the amount of extra data. This is also true for communication systems,where compression helps reduce the amount of bandwidth used which’s really important when we are streaming video or

sending data in real time. In these situations data is always coming in. The algorithms have to process it quickly without needing to look at it more than once.

When we have all the data already we can use something called Huffman coding,which makes a special coding tree based on how often each symbol appears. When we are dealing with live data like a video or network the symbols can change over time. That is where adaptive Huffman coding comes in and updates the coding tree as new data arrives making it better for real time applications.

Overall these compression techniques are a part of how we handle data efficiently in modern computers; they connect the ideas of Shannon entropy to the actual software and hardware we use. Huffman coding and entropy based compression techniques are essential in the way we communicate and store information,like test and images and audio and video using lossless compression algorithms.including Huffman coding to make files smaller without losing any information .

3. Methodology

3.1 Search Strategy *(by Samiul Hasan Bhuiyan & Md. Habibul Islam)*

Databases searched:

- IEEE Xplore (ieeexplore.ieee.org)
- ACM Digital Library (dl.acm.org)
- Google Scholar (scholar.google.com)
- arXiv (arxiv.org)
- MDPI (mdpi.com)
- SCIRP Open Access (scirp.org)
- arXiv (arxiv.org)

Search strings used:

Search string 1: “Huffman coding”

Search string 2: “Shannon entropy”

Search string 3: “adaptive Huffman” OR “Dynamic Huffman”

Search string 4: allintitle: Huffman complexity

Search string 5: “streaming”

Search string 6: “entropy coding”

Search string 7: "compression ratio"

Search string 8: “lossless compression”

3.2 Inclusion and Exclusion Criteria (*by Samiul Hasan Bhuiyan*)

Inclusion Criteria	Exclusion Criteria

• Published in peer-reviewed IEEE / ACM / Springer / Elsevier venue • Published between 2013 and 2025 • Discusses at least one algorithm relevant to your RQs • Written in English with full text accessible • Reports time or space complexity of the algorithm	• Blog posts, Wikipedia, Medium articles • Papers without algorithmic complexity discussion • Papers not accessible in full text • Papers published before 2010 (unless foundational classics)

3.3 Paper Selection Summary *(by Md. Habibul Islam & Samiul Hasan Bhuiyan)*

Total papers found in initial search	374
Papers removed after title/abstract screening	293
Papers removed after full-text review	24
Final papers included in this review	20

4. Review and Analysis

4.1 Static Huffman Coding *(by Abu Talha Swargo)*

Static Huffman coding is a technique used to compress data by assigning shorter code to symbols that appear often and longer codes to those that appear less frequency [1][2][3]. It creates a binary tree structure to represent these codes,which helps reduce the file size efficiently.The algorithm runs in O(nlogn)[1][2],where n is the number of distinct symbols[1]. It is widely used in formats like JPEG images,MP3 audio and ZIP files. The main drawback is that it needs two passes over the data[1]. First,the symbol has to scan the entire file to count how often each symbol appears, and then it processes the file again to apply the encoding.Because of this,it can be slow for very large files and it is not suitable for real-time or streaming down.

4.2 Adaptive Huffman Coding *(by Abu Talha Swargo)*

Adaptive Huffman coding improves on the static version by building the tree while the data set is being processed [2]. It does not need to know the frequencies at first. Instead, it updates the structure of trees every time a new symbol arrives. This achieves a time complexity of O(nlogn), where k is the message length and L is the tree depth [1][2].This algorithm is very useful and efficient for real-time tasks like live video streaming and network communication [2][4]. However, its key limitation is that

it uses a lot of CPU power. Because the tree is updated for every single symbol, it can be too slow for very high-speed data transfers.

4.3 Comparison Table *(by Md. Habibul Islam)*

Algorithm Name	Time Complexity	Space Complexity	Application	Key Limitation	Reference
Static Huffman Coding	O(nlogn)	O(n)	Standard file compression (ZIP), Image (JPEG) and Audio (MP3), Genomics, Neural networks	Two Pass: Must scan data twice,once to calculate frequencies and once to encode.	[1][8][11]
Adaptive Huffman Coding (Dynamic Huffman)	O(nlogn) per symbol	O(n)	Real-time sensor data,live streaming,and network protocols	live streaming, and network protocols.Computational Cost: Heavy CPU usage due to tree updates and rebalancing for every symbol processed.	[2]

4.4 Summary of Findings (*by Samiul Hasan Bhuiyan*)

As shown in Table 1, Shannon's entropy, introduced by Claude Shannon, defines the theoretical lower bound for lossless data compression. It represents the minimum number of bits required to encode data without losing information. Huffman Coding is designed to approach this limit by assigning shorter codes to more frequent symbols and longer codes to less frequent ones. As a result, Huffman coding achieves compression that is very close to the entropy bound, although it may slightly exceed it due to the use of whole (integer) bit lengths.

Huffman coding is widely used in traditional applications such as JPEG, MP3, and ZIP, and has also found use in modern areas like genomics and neural network compression; however, static methods require prior knowledge and a two-pass process, while adaptive methods incur higher computational overhead due to continuous tree updates.

Static Huffman coding performs best when symbol probabilities are known in advance and remain stable throughout the data, producing near-optimal compression with relatively low computational effort after initial tree construction. In contrast, adaptive Huffman coding dynamically updates the tree

as symbols are processed, making it suitable for real-time and streaming scenarios where data distributions may change.

From a complexity perspective, there is a clear balance between efficiency and adaptability. Static Huffman coding builds the tree once with a time complexity of O(n log n), while adaptive Huffman coding requires O(log n) time per symbol due to continuous updates, increasing overall computational cost for large datasets.

In relation to Research Question 1 (RQ1), Shannon's entropy establishes the optimal compression limit, and Huffman coding satisfies the condition $H(X) \leq L < H(X) + 1$, confirming that it achieves near-optimal performance. For Research Question 2 (RQ2), the optimality of Huffman coding is ensured by its greedy strategy, which always combines the least probable symbols first, guaranteeing the minimum average code length among all prefix-free coding schemes.

5. Discussion (*by Jobaer Abdullah*)

This section brings together what we found from reviewing past studies and highlights gaps in research. We focus on answering a question about balancing compression efficiency with computational cost in adaptive methods.

One major finding is that static Huffman coding works well almost as well as the best possible method predicated by Shannon entropy. This is especially true when we know how often each symbol appears beforehand.Huffman coding is good because its built in a way.However it needs two steps,which makes it less useful for a real-time system.

Adaptive Huffman coding solves this problem by updating the coding tree as it goes. This lets it work with streaming data without needing to know how often symbols appear . so,it's great for sending data in time and for network protocols.

Studies keep showing that adaptive Huffman coding uses a lot more computer power. We have to update and rebalance the tree after each symbol,which increases CPU use and delay. This confirms that while adaptive methods are more flexible they use resources. This directly answers our question.

New methods like the Asymmetric Numeral System(ANS) introduced by Jarek Duda in 2013 shows that we can get close to Shannon entropy while using computer power. ANS is now used in modern compression standards because it is faster and compresses better than traditional Huffman coding.

Variable-to-variable Huffman Coding tries to improve coding efficiency by encoding blocks of symbols of individual ones. This offers compression in the same cases but is more complicated to design.

5.1 Research Gaps Identified *(by Md. Habibul Islam)*

Research Gap 1 — Coverage Gap: Low-Power IoT Devices

Almost all the papers reviewed in this study tested Huffman coding on regular computers or large servers. These machines have strong processors, plenty of memory, and a reliable power source. But the real world also includes very small devices — like IoT sensors used in smart homes, health monitors, or environmental tracking systems.

The gap is this: no study has tested how Adaptive Huffman coding performs on these tiny, low-power IoT devices [6][18]. These devices have very limited battery life and very small memory. Adaptive

Huffman coding updates its tree structure every time a new symbol arrives — this process uses CPU cycles and therefore consumes power [2][6]. On a normal computer, this is not a big problem. But on a small sensor running on a battery, even small extra power usage can drain the battery quickly and make the system useless.

So the research community has ignored this important practical situation. We do not know if Adaptive Huffman coding is even usable on IoT devices, or if it is too heavy for them [6][18]. This is a real and serious gap because IoT devices are becoming very common in modern technology.

Research Gap 2 — Comparison Gap: Adaptive Huffman vs. ANS

A newer compression method called Asymmetric Numeral Systems (ANS) was introduced by Jarek Duda in 2013 [5]. ANS is now used in modern compression tools because it can reach very close to Shannon's entropy limit while also being faster than traditional Huffman coding [5]. Big companies like Google and Apple use ANS in their systems today [5].

The gap here is: there is no direct, side-by-side comparison between Adaptive Huffman coding and ANS on mobile phones or real-time applications [2][5]. The reviewed papers discuss each method separately — some papers talk about Adaptive Huffman, other papers talk about ANS — but no paper puts them together in a controlled test and measures which one is actually better in practice [2][5].

Because they are only studied separately, we cannot clearly answer which method is more suitable for real-time apps on smartphones — in terms of compression ratio, speed, and power use [5][7]. A head-to-head comparison on the same device and same data is completely missing from the current literature [5][7]. Without this, developers cannot make an informed choice about which algorithm to use.

Research Gap 3 — Recency Gap: GPU Acceleration

Most of the papers reviewed in this study are several years old and focus only on CPU performance [19][20]. They measure how fast Huffman coding runs on a central processor. However, modern computers, laptops, and even smartphones now have powerful GPUs (Graphics Processing Units). GPUs are designed to handle many small tasks at the same time — this is called parallel processing — which makes them very fast for certain types of work [16].

The gap is: no recent study has tested whether a GPU can speed up the Huffman tree-update process, especially for demanding tasks like 4K live video streaming [16][7]. The existing time complexity numbers — like O(nlogn) for Static Huffman or O(nlogn) for Adaptive Huffman — were all measured based on single-processor CPU behaviour [1][2]. These numbers may no longer be accurate or relevant when modern GPU hardware is used [16].

In other words, the whole performance picture of Huffman coding has not been updated for today's hardware [16][7]. Researchers and engineers who work with modern systems cannot rely on old CPU-based results to understand how these algorithms truly perform now [19][20]. This is a significant gap because GPU usage is now standard, not exceptional.

5.2 Answers to Research Questions (*by Samiul Hasan Bhuiyan*) **RQ1**
Answer:

Shannon's entropy, introduced by Claude Shannon, provides a theoretical lower bound on the average number of bits required to encode symbols in any lossless data compression system. It is defined as:

$$H(X) = -\sum_{i=1}^{n} pi \cdot log2\ pi$$

the inherentpi uncertainty or information content in a source and establishes the optimal compression where represents the probability of occurrence of the ithi^{th}ith symbol. This measure quantifies limit that no encoding scheme can surpass.[1][2][7]

Huffman coding attempts to approach this theoretical limit by assigning variable-length binary codes to symbols based on their probabilities. Symbols with higher probabilities are assigned shorter codewords, while less frequent symbols receive longer codes. The average code length achieved by Huffman coding is expressed as: $L = \sum_{i=1}^{n} pi \cdot li$

where li is the length of each code.[1][2][9]

A fundamental property of Huffman coding is that its average code length satisfies the inequality:

$H(X) \leq L < H(X) + 1$

This demonstrates that Huffman coding is nearly optimal, achieving compression efficiency very close to the entropy limit. However, due to the restriction of using integer-length codewords, it may slightly exceed the theoretical minimum.[1][2][4][10]

In practical applications, Huffman coding performs best when symbol probabilities are known in advance and remain relatively stable. In dynamic or unpredictable environments, its efficiency may decrease, as the coding scheme cannot adapt instantly to changes in symbol distribution.

RQ2 Answer:
Huffman coding is optimal because it minimizes the average code length:

$$= \sum_{i}^{n} pi \cdot \quad L$$

$=1 \qquad di$ is

di

where the depth of each symbol in the tree.

The algorithm follows a greedy approach, repeatedly combining the two symbols with the smallest probabilities to construct a binary tree in a bottom-up manner. This approach is proven to be optimal based on several theoretical properties.

First, the greedy choice property ensures that in an optimal prefix-free code, the two least probable symbols must appear as sibling leaves at the maximum depth of the tree. Any deviation from this arrangement would result in a higher expected code length, contradicting optimality.

Second, the problem exhibits optimal substructure. By merging the two least probable symbols into a composite symbol, the problem is reduced to a smaller instance. Solving this reduced problem optimally guarantees that the overall solution remains optimal.

Finally, the correctness of the algorithm is supported by an exchange argument. Any non-optimal tree structure can be transformed into an optimal one by rearranging nodes without increasing the total cost. This confirms that the greedy approach used in Huffman coding always produces a minimum expected code length.

RQ3 Answer:

Adaptive Huffman coding extends the traditional static approach by dynamically updating the coding tree as data is processed. Unlike static Huffman coding, which requires prior knowledge of symbol probabilities, adaptive methods build and adjust the tree in real time, making them suitable for streaming data and unknown input distributions.

As each symbol is read, the tree structure is modified to reflect updated frequencies. This allows the encoding process to gradually improve its efficiency as more data becomes available. Such adaptability is particularly beneficial in applications where data characteristics change over time.

However, this flexibility introduces additional computational overhead. Each symbol insertion may require updating node weights and performing tree adjustments, including node swaps to maintain the required structure.

In terms of computational complexity, static Huffman coding constructs the tree once with a time complexity of O(nlog n) [1][2], where nnn is the number of distinct symbols. In contrast, adaptive Huffman coding requires O(log n) [1][2] time per symbol due to continuous updates. Despite these differences, both methods have a space complexity of O(n) [1][2][5][8].

Overall, adaptive Huffman coding is more suitable for dynamic and unpredictable data environments, whereas static Huffman coding is more efficient when symbol probabilities are known and fixed.

6. Conclusion (by Jobaer Abdullah)

This paper is about Shannon’s Entropy and Huffman Coding. These two things are really important for making data smaller. Shannon’s Entropy tells us the size that data can be made. This is the standard that we use to see how good our coding techniques are. Huffaman Coding is the way to make data smaller that is actually pretty close to the size possible.

Our main contribution is the comparison table and the gap analysis. The table shows that Adaptive Huffman is more flexible but uses more CPU power. We found that more research is needed for new technologies like GPUs and IoT devices.

One of the things that this study did was make a table that compares the different ways of doing things. This table shows the bad things about each method. What we found out is that the methods we have now are good. They can be made better. Now technologics like using graphics cards and internet devices are giving us challenges and chances to make compression better.

Future Work (by Jobaer Abdullah)

Future studies should look at how to make data compression faster using powerful computer parts like GPUs. Researchers need to see if the main problem is the speed of updating data or the memory.Also we need to test how well these methods work when information changes quickly or is hard to predict. Finally,we should compare old and new ways of compressing text,video and DNA data to find the best system for the future.

Graphics cards are really good at handling a lot of things at the time which makes them perfect for parts of compression that can be done in parallel . But we have to figure out what is slowing things down. It is the work of how we are getting to the information . If we look closely at these things we can make compression systems that work better with the hardware.

We also need to see how compression works when things are changing fast . Huffman coding works best when things are stable but that does not always happen in real life.For example sensor networks ,financial data and live video feed are always changing. So we need to make compression methods that can change quickly without using much extra power.

Another thing we should do is compare new compression methods with different kinds of data. We should look at methods like Huffman coding and machine learning based compressors. We need to think about more than how much we can compress the data . We also need to think about how fast it is, how much memory it uses and how well it works on devices that do not have a lot of power like Internet of Things devices.

Lastly we should try different compression methods,with different types of data like text, high quality video and DNA sequences . Each type of data is unique so a method that works perfectly with one type might not work with another . If we can make systems that can adapt to types of data we can make compression technologies that will work perfectly in the future. Data compression is really important in real-life. We need to make sure we are looking at all the different ways we can make it better. Data compression methods need to be able to handle all kinds of data and data compression systems need to be able to work with all kinds of hardware.

Appendix A. AI Prompt Log

Date	Member	AI Tool	Section	Prompt (summary)	AI Output (summary)	Verified?
April 13 2026	Md. Habibul Islam	Gemini	2	What is Shannon entropy and adaptive Huffman coding?	Shannon entropy defines the theoretical limit of lossless compression by measuring data's average uncertainty, while adaptive Huffman coding is a real-time compression algorithm that dynamically builds its encoding tree as it processes data, rather than requiring a pre-calculated frequency table.	No
April 16 2026	Md. Habibul Islam	ChatGPT	2.1	Why is Huffman coding called a greedy algorithm? What are the two main properties needed to prove it works?	Explained the greedy choice of picking the two smallest frequencies to build the tree upwards.	No
April 16 2026	Abu Talha Swargo	ChatGPT	4.2	I need to know if Adaptive Huffman is better for streaming because it only reads data once.	Confirmed it uses a single pass and updates the tree instantly using the Sibling Property.	Yes[2]
April 16 2026	Md. Habibul Islam	Perplexity	1.3	I need help writing research questions that compare compression speed and theoretical limits.	Suggested RQs focusing on entropy bounds, greedy optimality, and CPU costs for streaming.	No

April 17 2026	Md. Habibul Islam	Perplexity	5.1	I am looking for research gaps. What is missing in modern studies about Huffman coding?	Identified gaps in GPU speed, IoT battery life, and comparisons with new methods like ANS.	No

Date	Member	AI Tool	Section	Prompt (summary)	AI Output (summary)	Verified?
April 10 2026	Md. Samiul Hasan Bhuiyan	ChatGPT	4.3	<Full topic P11-A> Explain this topic and tell me its key points and what the true purpose of the research topic is.	Explained the topic problem, explained the RQ's. Highlighted key points such as Shannon's entropy, Huffman Coding algorithm, and also mentioned that the purpose of this research topic is to compare static vs dynamic huffman coding and computational overhead that comes together with dynamic version.	No
April 10 2026	Md. Samiul Hasan Bhuiyan	ChatGPT	3.1	Give me search strings to find papers related to this topic from databases like Google Scholar and IEEE	Gave several sample strings to find related papers	No
April 12 2026	Md. Samiul Hasan Bhuiyan	ChatGPT	4.3, 4.4 5.2	Create a table template for spreadsheet where we can store paper info to later use as reference.	Created a table template to copy in google sheet and keep track of papers	No

April 17 2026	Md. Samiul Hasan Bhuiyan	ChatGPT	4.4, 5.2	<Bullet point answer of the RQ's> Rewrite these in paper writing format and expand the answers if necessary	Converted the bullet point answers of RQ's to paper writing format	Yes [1][2]
April 16 2026	Jobaer Abdullah	ChatGPT	2.3	Explain real-life applications of Huffman coding and entropy in simple terms for a project report.	Explained how Huffman coding used in file compression,communication system and real -time streaming with examples.	Yes [1][10]

Date	Member	AI Tool	Section	Prompt (summary)	AI Output (summary)	Verified?
April 17 2026	Jobaer Abdullah	ChatGPT	5	Help to write a discussion about comparing static vs adaptive huffman coding and identify research gap	Provided a comparison of both methods,discussed and suggested gap like IoT devices,GPU acceleration,and ANS and comparison.	Yes [5][7]
April 17 2026	Jobaer Abdullah	ChatGPT	6	Help to write a sample conclusion and future work section for a paper on Huffman coding and Shannon entropy.	Generated a conclusion about entropy limits and Huffman coding performance and suggested future work on GPUs, IoT and modern compression methods.	Yes[5]

April 17 2026	Abu Talha Swargo	ChatGPT	1.1	Help to write a basic sample of motivation and background section for a paper on Huffman coding and Shannon entropy.	Modern data compression relies on Claude Shannon's entropy to define the theoretical limit and David A. Huffman's coding method to efficiently approach that limit by reducing data size without loss.	No
April 17 2026	Abu Talha Swargo	ChatGPT	1.2	Help to write down the problem statement about Huffman coding and Shannon entropy.	Adaptive Huffman improves streaming compression over static but with extra computational cost, balancing efficiency and speed near Claude Shannon's limit.	No
Date	**Member**	**AI Tool**	**Section**	**Prompt (summary)**	**AI Output (summary)**	**Verified?**
April 17 2026	Abu Talha Swargo	ChatGPT	4.1	I want to know that Static Huffman coding is a technique used to compress data by assigning shorter code?	Static Huffman coding reduces data size by encoding common symbols with fewer bits and rare symbols with more bits based on predetermined frequencies.	No

References. *(by Samiul Hasan Bhuiyan & Md. Habibul Islam)*

[1] S. Pigeon, "Huffman coding," in Data Compression Chapter,

[2] T. Gagie, "Dynamic Shannon coding," arXiv preprint, 2005.

[3] M. Al-Kuwari et al., "Variable-to-variable Huffman coding," Entropy, vol. 24, no. 10, p. 1447, Oct. 2022. doi: 10.3390/e24101447.

[4] M. H. Btoush, "A complexity analysis and entropy for different data compression algorithms on text files," Journal of Computer and Communications, vol. 6, no. 10, Oct. 2018. doi:

10.4236/jcc.2018.61029.

[5] J. Duda, “Asymmetric numeral systems,” arXiv preprint, Nov. 2013. doi: 10.48550/arXiv.1311.2540.

[6] S. Pigeon, “A memory-efficient adaptive Huffman coding algorithm for very large sets of symbols revisited,” Tech. Rep., Université de Montréal, Montréal, Canada, 1997.

[7] S. T. Klein, “Forward looking Huffman coding,” Theory of Computing Systems, 2020. doi: 10.1007/s00224-020-09992-7.

[8] H. Chen, "Genome sequence compression based on entropy coding with context quantization and spatial co-location pattern," in Proc. ACM Conference, 2025. doi: 10.1145/3747227.3747232

[9] D. A. Huffman, “A method for the construction of minimum-redundancy codes,” Proceedings of the IRE, vol. 40, no. 9, pp. 1098–1101, Sep. 1952. doi: 10.1109/JRPROC.1952.273898.

[10] C. E. Shannon, "A mathematical theory of communication," The Bell System Technical Journal, vol. 27, no. 3, pp. 379–423, July 1948, doi: 10.1002/j.1538-7305.1948.tb01338.x.

[11] E. Tartaglione and S. Lathuilière, "High-order entropy minimization for neural network compression," Neurocomputing, vol. 461, pp.244-253, 2021. doi: 10.1016/j.neucom.2021.07.022

[12] D. Salomon, Data Compression: The Complete Reference, 4th ed. London, U.K.: Springer, 2011. doi: 10.1007/978-1-84882-903-9

[13] T. M. Cover and J. A. Thomas, Elements of Information Theory, 2nd ed. Hoboken, NJ, USA: Wiley, 2012. doi: 10.1002/047174882X

[14] H. Zenil et al., "Dynamical complexity of time series," arXiv preprint arXiv:1609.01924, 2016.

[15] Z. Zhang et al., "Semantic Huffman coding," arXiv preprint arXiv:2401.14634, 2024.

[16] B. Johnston and E. McCreath, "Parallel Huffman Decoding: Presenting a Fast and Scalable Algorithm for Increasingly Multicore Devices," 2017 IEEE International Symposium on Parallel and Distributed Processing with Applications and 2017 IEEE International Conference on Ubiquitous Computing and Communications (ISPA/IUCC), Guangzhou, China, 2017, pp. 949-958, doi: 10.1109/ISPA/IUCC.2017.00146

[17] K. Sayood, Introduction to Data Compression, 5th ed. Cambridge, MA, USA: Elsevier, 2017. doi: 10.1016/C2015-0-01371-8

[18] P. Fenwick, "Adaptive Huffman coding revisited," IEEE Journal, 2013.

[19] "Efficient Huffman tree construction," in Proc. IEEE Conference, 2015.

[20] "Cache-efficient Huffman coding," in Proc. ACM Conference, 2017.